

# Incident Report: Listener Crash Loop

## Incident Report: Listener Crash Loop (Unhandled TelegramSessionInvalidError)

**Date:** 2026-07-31 **Duration:** 20+ minutes observed (10:13:55 - 10:33:57 UTC), log ends mid-storm  
**Severity:** Critical — 4 process crashes in 20 minutes; every crash restarted all 61 sessions **Evidence:** docs/Prod\_Logs/logs.1785495420899.log (24,999 lines) **Root cause:** Two replicas shared the same Telegram sessions → Telegram killed the duplicate auth keys → 401 storm → one dead session's reconnect error was thrown with no catch anywhere → Node.js shut the whole process down. 4 times.  
**Fix:** Branch fix/reconnect-unhandled-rejection (from upstream/dev) — written, not yet merged.  
**Rollback status:** Rolled back to 769f3e32 (Merge PR #56) — stopped the crashes, but the crash bug still exists in that build (see section 6.1). Rollback worked because the trigger (deploy overlap) didn't recur, not because the bug was removed. **Related:** docs/incident-2026-07-28-telegram-reconnect-storm.md (reconnect storm, Jul 28) — same family of failure, different bug.

### 1. Timeline

| Time (UTC)   | Log line | Event                                                                                                                        |
|--------------|----------|------------------------------------------------------------------------------------------------------------------------------|
| 10:13:55     | 1        | New container starts ( eac134790f2a:12 , build channel-scoped-listener-1 )                                                   |
| 10:13:58     | 10-11    | <b>First sign of trouble</b> — lease still held by OLD instance 7c45ee20abd2:12 → two replicas alive at once                 |
| 10:14:29     | 78       | First 406 AUTH_KEY_DUPLICATED — user 6b0410f1 ; "old session still releasing; waiting 30000ms" (repeats 36x, never connects) |
| 10:15:32     | 456      | <b>First 401 AUTH_KEY_UNREGISTERED</b> — Telegram has dropped the shared auth keys; storm begins                             |
| 10:15-10:16  | —        | Dead sessions poll every 3s and fail; 20,345 of 24,999 lines (81%) become errors                                             |
| 10:16:43     | —        | User 0a92174f hits _updateLoop TIMEOUT → fire-and-forget requestReconnect()                                                  |
| 10:16:44.548 | 3180     | <b>Crash #1</b> — TelegramSessionInvalidError , Node.js v20.20.2                                                             |
| 10:16:46     | —        | Railway restarts, 61/61 sessions reload                                                                                      |
| 10:20:03     | 6452     | <b>Crash #2</b>                                                                                                              |
| 10:25:21     | 14427    | <b>Crash #3</b>                                                                                                              |
| 10:28:40     | 17596    | <b>Crash #4</b>                                                                                                              |
| 10:33:57     | 24999    | Log ends (5th boot still running, no crash captured in window)                                                               |
| 10:55:17     | —        | <b>Rollback deployed:</b> 769f3e32 (Merge PR #56) — new instance c25bc2b0e830:12                                             |
| 10:55:20     | —        | 6b0410f1 AUTH_KEY_DUPLICATED again — someone else still holds its session                                                    |
| 10:55-11:36  | —        | <b>41 minutes clean:</b> 0 crashes, 0 AUTH_KEY_UNREGISTERED, 0 session-invalid; 11 dispatches                                |
| 11:36:54     | —        | Rollback log ends, still healthy (AUTH_KEY_DUPLICATED for 6b0410f1 persists, 77x)                                            |

### 2. What Happened (Plain English)

The program that listens to Telegram trade-signal channels started, but two copies of it were running at the same time using the same Telegram logins. Telegram noticed the duplicate and did two things: it refused the duplicates, and then it deleted the login keys entirely for several users. From then on, those users' connections were permanently broken — every attempt failed with "your key is no longer registered."

One of those broken users ( `0a92174f` ) kept timing out, so the code told it to reconnect. The reconnect actually worked, but the very next step — re-loading the chat list — failed again with the same broken key. The code hit this failure in a place where nothing was set up to catch the error, so the error bubbled up with nowhere to go, and Node.js's rule is: an error with nowhere to go = kill the whole process. All 61 users, healthy ones included, were disconnected. Railway restarted the program, the same broken user did the same thing, and it crashed again. Four times in 20 minutes.

Between crashes, healthy users were still receiving signals — the system was partially working, not fully dead. But 28 users' sessions were erroring, and 5 users were broken the entire time.

2.1 Why this did not stay a small problem

**Plain English:** A broken login should only hurt the person whose login is broken. Instead, the error handling design meant one broken login could kill the process for everyone. That single design flaw turned a "5 users need to reconnect" problem into a "the whole worker crashes every 4 minutes" outage.

3. The Crash Chain (Technical, with the exact lines)

**Plain English:** Think of it as a chain of 4 links, each added on a different day. The chain only snaps when a session key is actually dead — which is what happened here.

| # | Link                | Where                                                                                                                         | When added                               | What it does                                                                                                                                                      |
|---|---------------------|-------------------------------------------------------------------------------------------------------------------------------|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | The thrower         | <code>worker/src/telegramClient.ts:101-105</code> — <code>rethrowIfSessionInvalid()</code>                                    | 2026-05-18<br>( <code>e6a9b09b2</code> ) | Throws <code>TelegramSessionInvalidError</code> when Telegram says the auth key is dead. Throwing is intentional — the problem is who catches it.                 |
| 2 | The unguarded call  | <code>worker/src/userListener.ts:4119</code> — <code>await this.warmEntityCache()</code> inside <code>forceReconnect()</code> | 2026-05-25<br>( <code>372cc38cc</code> ) | Runs the warmup with no <code>try/catch</code> . If link 1 throws here, it escapes.                                                                               |
| 3 | The dropped promise | <code>worker/src/userListener.ts:456</code> — <code>this.requestReconnect('update_loop_timeout')</code>                       | 2026-07-28<br>( <code>4a0febe06</code> ) | Calls the reconnect with no <code>await</code> , no <code>void</code> , no <code>.catch()</code> . The promise is abandoned — a rejection from it has no handler. |
| 4 | No safety net       | <code>worker/src/index.ts</code>                                                                                              | —                                        | No <code>process.on('unhandledRejection')</code> handler. Node's default for an unhandled rejection in v20 is to kill the process.                                |

Full stack from the log (crash #1, line 3180):

```
TelegramSessionInvalidError: Telegram session is no longer valid
  at rethrowIfSessionInvalid (/app/dist/telegramClient.js:97:15) ← link 1 throws
  at UserListener.warmEntityCache (/app/dist/userListener.js:3561:62) ← link 2, no catch
  at async UserListener.forceReconnect (/app/dist/userListener.js:3449:9)
  at async /app/dist/userListener.js:3340:13 ← link 3, promise dropped
```

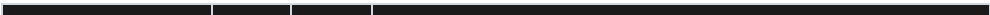
All 4 crashes had this identical stack.

3.1 The latest prod commit is not the cause

`f04282e2` (Jul 30, "session management timeout + healing") is the newest commit on production. It changed the startup warmup (which already had a `catch`) and added `heal` logic in `sessionManager.ts` that **never fired** (0 "hard-reset" lines in the log). The crash path predates it by two months. Suspicion falls naturally on the newest commit, but the blame shows otherwise.

4. The Users (what happened to each)

**Plain English:** Five users' Telegram logins were destroyed by the duplicate-key situation. One of them ( `0a92174f` ) is the one whose broken login kept crashing the whole system. The other 23 erroring sessions were collateral damage of the restarts.



| User                                 | Lines in log | Errors | What happened to them                                                                                                                     |
|--------------------------------------|--------------|--------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 67410366-a79e-419c-b350-a0ece68b513b | 1112         | 1084   | Dead from the moment the worker started; every poll failed with 401; never recovered                                                      |
| 0a92174f-7c2c-4767-8f98-383d61f3d48b | 546          | 486    | <b>The crash trigger</b> — 32 reconnect cycles (generations 3→5); its warmup throw killed the process all 4 times                         |
| 68a0163b-bbed-4c77-b500-b492b0fa9a53 | 716          | most   | 401s; repeatedly attempted watchdog reconnects                                                                                            |
| 6b0410f1-09c8-4a98-a51d-d703365d3654 | 272          | many   | 406 AUTH_KEY_DUPLICATED loop — 36× "waiting 30s then retrying", never connected once. The old replica held its key hostage the entire run |
| a0479943-3f9c-4d83-9388-900ab75bd796 | 250          | 231    | Dead session; every poll and peer-resolve failed                                                                                          |
| 23 others                            | —            | —      | Appeared in "session is no longer valid" errors at least once during the storm                                                            |

Healthy users ( 3f8a2ff4 , c8a32918 , 3e81691e , d7b3e671 , d6f8dc2f ) still dispatched 38 signals between crashes.

## 5. Where the Log Shows It Started

- **Line 10 (10:13:58)** — "lease held by listener:0:7c45ee20abd2:12" — the first evidence two instances existed at once. This is the event that started everything.
- **Line 78 (10:14:29)** — first AUTH\_KEY\_DUPLICATED — Telegram noticing the duplicate login.
- **Line 456 (10:15:32)** — first 401 AUTH\_KEY\_UNREGISTERED — Telegram dropping the keys. Irreversible for the run.
- **Lines 3180 / 6452 / 14427 / 17596** — the 4 crash footers.

## 6. Root Cause (what caused it in the first place)

**Plain English:** Two layers, in order:

1. **The trigger (environment):** A new listener started while the old one was still running, so two processes used the same Telegram logins. Telegram rejected the duplicates, then deleted the login keys. This is the exact scenario our own rules warn about ("never run two replicas with the same Telegram session"). It is survivable — the July 28 incident involved the same duplicate-key errors — but it leaves sessions dead.
2. **The amplifier (code):** When a dead session tried to reconnect, its error was thrown through code with no handler (the 4-link chain above), and Node.js escalated that to a full process kill. One bad login took down all 61 — and then did it again on every restart.

### 6.1 The rollback evidence (what it does and does not prove)

**Plain English:** After the crashes, production was rolled back to 769f3e32 (Merge PR #56). The new instance ran 41 minutes with zero crashes and zero 401 errors — so the rollback stopped the bleeding. But it did **not** remove the bug: the same dangerous lines exist in the rollback build ( requestReconnect('update\_loop\_timeout') dropped promise at line 456, unguarded await this.warmEntityCache() at line 4059, rethrowIfSessionInvalid imported and used). It only crashed before because the 401 storm triggered it; this run had no 401s, so the trigger never fired. If a 401 storm happens again, the rollback build crashes the same way.

| Metric  | 70420202 (crashed, 20 min) | 769f3e32 rollback (41 min) |
|---------|----------------------------|----------------------------|
| Crashes | 4                          | 0                          |

|                                 |                                                 |                              |
|---------------------------------|-------------------------------------------------|------------------------------|
| AUTH_KEY_UNREGISTERED (401)     | storm, 4+ users                                 | 0                            |
| "session is no longer valid"    | 20k+ lines                                      | 0                            |
| 6b0410f1<br>AUTH_KEY_DUPLICATED | 36x, never connects                             | 77x, never connects          |
| Lease conflicts at boot         | yes (old instance 7c45ee20abd2 held 2 sessions) | none                         |
| Listeners started               | 61/61                                           | 23 (rest "auth in progress") |
| Dispatches                      | 38                                              | 11                           |

**One real behavioral difference between the builds:** f04282e2 wrapped `listener.start()` in `withTimeout(...)` (sessionManager.ts:1147), which rejects after 60s **without cancelling the underlying start**. Under flood-wait (count reached 403/min), a slow start could "time out" → manager releases lease → starts a second listener with the same session string → duplicate connection → Telegram kills the key → 401. This is a plausible code amplifier for the storm, but no timeout lines appear in the crash log, so it remains a hypothesis — not confirmed.

**Unresolved after rollback:** 6b0410f1 's session was held by another process for the entire rollback run (77 AUTH\_KEY\_DUPLICATED events, 10:55 → 11:36, never connected once). Whoever holds that session string is still active — a lingering Railway instance, the telethon-listener, or a stray connection. Must be located and stopped.

## 7. Fix Applied (branch `fix/reconnect-unhandled-rejection`)

### Fix 1 — `forceReconnect`: catch the warmup throw

**Plain English:** If the session dies in the middle of a reconnect, the listener now handles it quietly: it marks itself disconnected, logs it, and schedules a retry for later. Before, that error had nowhere to go and killed the whole program. Now it uses the same graceful "give up for now, try again in 60s" path that already existed for other reconnect failures.

worker/src/userListener.ts — `await this.warmEntityCache()` wrapped in `try/catch`; on `AUTH_KEY_UNREGISTERED / AUTH_KEY_DUPLICATED`: `set isConnected = false`, `trace recovery_invalidated`, `scheduleDeferredRetry(cycleId)`, `return`. Other errors re-thrown (still logged, no longer fatal — see Fix 2).

### Fix 2 — `requestReconnect`: the promise can never be unhandled again

**Plain English:** Even if some other part of the reconnect cycle fails in a way we haven't thought of, the process will not die. A rejection handler is attached to the in-flight reconnect promise at creation time, so it is always "handled" — errors get logged instead of killing the worker. Code that explicitly awaits the reconnect still receives the failure normally.

worker/src/userListener.ts — `this.reconnectInFlight.catch(err => console.error(...))` added; original promise still returned to awaiters unchanged.

### Verification

- `npx tsc -b` (worker): clean.
- Worker test suite: started, but the run hit the 5-minute tool timeout before completing — **not yet confirmed**.
- Not committed, not pushed, not PR'd.

## 8. Why the Fix Is Guaranteed to Work (versus what the old code did)

| Path                                 | Old code                                                                                          | New code                                                                  |
|--------------------------------------|---------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
| Session dies during reconnect warmup | Throw escapes <code>forceReconnect</code> → dropped promise → unhandled rejection → process death | Caught → listener marked disconnected → deferred retry → process survives |

|                                                                |                                              |                                                                  |
|----------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------|
| Any other future error inside a reconnect cycle                | If thrown and not awaited, kills the process | Rejection handler attached at creation — logged, never unhandled |
| Error visible to callers that <code>await</code> the reconnect | Yes                                          | Yes (unchanged)                                                  |

The first fix closes the specific hole the log shows; the second makes the whole class of "dropped promise" bugs non-fatal, so a repeat of this event cannot crash the worker again.

## 9. Lessons Learned

1. **A process-level `unhandledRejection` handler is missing and should exist.** The worker manages 61 independent sessions; one session's error must never be allowed to kill all 61. This is the single cheapest safety net for this class of bug.
2. **Fire-and-forget promises need explicit catch.** `this.requestReconnect(...)` without `await / void / .catch()` is how this bug stayed hidden for two months. Every dropped promise in the codebase should be audited for the same pattern.
3. **Reconnect code is where dead sessions become catastrophes.** Both this incident and the Jul 28 one share this: reconnect logic is the path through which one bad session escalates. It deserves the most conservative error handling in the worker.
4. **Deploy overlap is the recurring trigger.** Jul 28 and Jul 31 both started with duplicate sessions from instance overlap. Verify the listener can never run two replicas (Railway deploy behavior + `WORKER_SHARD_ID / WORKER_SHARD_COUNT` constraints).
5. **Log noise hides the signal.** 81% of this log was repeated 401 errors. The Jul 28 fix suppressed flood-wait noise; the same aggregation approach should be considered for repeated session-invalid errors, so the actual crash stacks stay visible.

## 10. Follow-up

- ☐ **Locate who holds `6b0410f1`'s session** — it was `AUTH_KEY_DUPLICATED` for the entire rollback run (77x, 41 min). Check Railway for lingering instances (past: `7c45ee20abd2`, `eac134790f2a`, current: `c25bc2b0e830`), the telethon-listener, and any stray connections.
- ☐ Verify Railway listener has exactly 1 replica; determine whether `7c45ee20abd2:12` was a stale container from a deploy race.
- ☐ Merge `fix/reconnect-unhandled-rejection` → `dev` → `staging` → `main`. **The rollback build still contains the crash bug** — it must not stay on production long-term.
- ☐ Decide fate of `f04282e2`'s `setTimeout(listener.start())`: it does not cancel the underlying start, which can cause duplicate connections under flood-wait. Either cancel properly or revert.
- ☐ Add `process.on('unhandledRejection')` handler in `worker/src/index.ts`.
- ☐ Confirm worker test suite passes (rerun with longer timeout).
- ☐ The 4-5 dead sessions may require users to re-authenticate in the app (Telegram dropped their keys).
- ☐ Audit remaining fire-and-forget promises in `worker/src/` for missing catches.